Guía de estudio · Parcial 2

Programación Orientada a Objetos · 2027-1 · examen el miércoles 18 de noviembre de 2026, en el horario del laboratorio

> **Cómo usar esta guía.** Tiene la misma forma del examen. Resuelve cada parte en papel y después comprueba tu código en tu IDE. Esta guía no trae las respuestas: si algo no te sale, regresa a la práctica o a la clase que dice la tabla de temas.

1. Cómo es el examen

Parte	Qué te pide	Puntos
A	20 preguntas de opción múltiple. La mayoría son de los pilares (encapsulamiento, herencia, polimorfismo, abstracción e interfaces) y de excepciones; unas pocas son de la base de Java.	40
B	Un diagrama UML como los de las prácticas y 6 preguntas abiertas sobre él. Contestas con tus palabras, en una a tres oraciones.	30
C	Escribir código sobre el mismo diagrama: una clase hija completa y un `for` con polimorfismo.	30

Se contesta en el portal del examen, igual que el Parcial 1. Viene de todo lo visto en el curso, con el peso puesto en la programación orientada a objetos.

2. Qué estudiar

Tema	Lo que debes poder explicar y usar	Dónde repasarlo
Encapsulamiento	private, public y protected: quién ve qué getters y setters; validar en la puerta final en atributos this para distinguir atributo de parámetro	S16, S18 · P9
Herencia	extends: la hija es un tipo del padre super(...) y el orden de los constructores @Override y super.metodo() clases abstractas y métodos abstractos; final en clases	S16, S18 · P7, P8
Polimorfismo	variable del tipo padre con objeto de la hija enlace dinámico: el objeto decide qué método corre el for sobre un arreglo del tipo general, sin if por tipo sobrescribir contra sobrecargar; cast e instanceof	S20 · P10, P11

Interfaces y abstracción	interface e implements: un contrato de lo que la clase sabe hacer una clase extiende una sola clase e implementa varias interfaces ni las interfaces ni las abstractas se crean con new clase abstracta (es un) contra interfaz (sabe hacer)	S20, S23 · P11
Excepciones	qué es una excepción y cómo leer la traza try, catch y finally: qué corre y qué se salta throw new IllegalArgumentException(...) para rechazar un dato las comunes: NullPointer, ArrayIndexOutOfBounds, NumberFormat, InputMismatch, Arithmetic	S23
Leer UML	los tres compartimentos de la caja; +, -, # línea continua con triángulo: extends línea punteada con triángulo: implements cursiva: abstracto	S18 · P8, P10, P11
Base de Java (U1)	tipos, arreglos, if, switch, for, while return contra imprimir referencias y null	S03 a S14 · P1 a P6

Los tres errores que más puntos costaron en el Parcial 1.

1. **Imprimir en lugar de devolver.** Si el enunciado dice «devuelve», el método termina con `return`, no con `System.out.println`.
2. **Dejar el código en blanco.** Una clase con la cabecera, el constructor y el método a medias vale puntos. En blanco vale cero.
3. **Describir en lugar de contestar.** Si te preguntan qué imprime, escribe la salida exacta. Si te preguntan por qué, da la razón, no repitas la pregunta.

3. Práctica de la Parte A · opción múltiple

Elige una sola opción y anota por qué las otras tres no son.

1. **Herencia.** ¿Qué imprime new B(); ?

```
class A { A() { System.out.print("A"); } }
class B extends A { B() { System.out.print("B"); } }
```

   A. B
   B. BA
   C. AB
   D. No compila

2. **Herencia.** Padre solo tiene el constructor Padre(int x). ¿Qué pasa con esta clase?

```
class Hija extends Padre {
    Hija() { }
}
```

   A. Compila: Java llama a Padre(0)
   B. No compila: falta llamar a super(...) con un int
   C. Compila, pero truena al crear una Hija
   D. No compila: una hija no puede tener constructor

3. **Polimorfismo.** Perro sobrescribe sonido() para devolver "Guau". ¿Qué imprime?

```
Animal a = new Perro();
System.out.print(a.sonido());
```

   A. Guau
   B. ...
   C. No compila
   D. Truena al ejecutar

4. **Polimorfismo.** ladrar() solo existe en Perro. ¿Qué pasa?

```
Animal a = new Perro();
a.ladrar();
```

   A. Ladra, porque el objeto es un Perro
   B. Truena al ejecutar
   C. Compila y no hace nada
   D. No compila: la variable es de tipo Animal y Animal no tiene ladrar()

5. **Polimorfismo.** ¿Cuál es la diferencia entre sobrescribir y sobrecargar un método?

    A. Son lo mismo
    B. Sobrescribir: mismo nombre y otros parámetros en la misma clase. Sobrecargar: misma firma en la hija
    C. Sobrescribir: la hija vuelve a escribir un método del padre con la misma firma. Sobrecargar: mismo nombre con otros parámetros
    D. Sobrescribir solo se puede con métodos static

6. **Abstracción.** Figura es una clase abstracta. ¿Qué pasa con Figura f = new Figura(); ?

    A. No compila: una clase abstracta no se puede instanciar
    B. Compila y crea una figura vacía
    C. Truena al ejecutar
    D. Compila solo si Figura no tiene métodos abstractos

7. **Interfaces.** ¿Cuál declaración es válida en Java?

    A. class Pato extends Ave, Nadador
    B. class Pato implements Ave extends Nadador
    C. class Pato extends Volador implements Ave (con Volador interfaz y Ave clase)
    D. class Pato extends Ave implements Volador, Nadador

8. **Interfaces.** Volador declara void volar(); ¿Qué pasa con esta clase?

```
class Dron implements Volador {
    void volar() { }
}
```

    A. Compila sin problema
    B. No compila: el método debe ser public, como en la interfaz
    C. No compila: falta la palabra abstract
    D. Compila, pero volar() no se puede llamar

9. **Encapsulamiento.** saldo es private en Cuenta. ¿Cómo lo consulta la clase CuentaAhorro, que hereda de Cuenta?

    A. Con this.saldo
    B. Con super.saldo
    C. Con getSaldo(), si Cuenta lo ofrece
    D. No hay forma

10. **Encapsulamiento.** ¿Para qué sirve validar dentro de un setter o de un constructor?

    A. Para que el programa corra más rápido
    B. Para que el objeto nunca quede en un estado inválido, lo use quien lo use
    C. Para poder heredar de la clase
    D. Para no tener que escribir getters

11. **Excepciones.** ¿Qué imprime?

```
try {
    int[] a = new int[2];
    a[2] = 1;
    System.out.print("A");
} catch (ArrayIndexOutOfBoundsException ex) {
    System.out.print("B");
} finally {
    System.out.print("C");
}
```

    A. ABC
    B. AC
    C. B
    D. BC

12. **Excepciones.** ¿Qué excepción lanza Integer.parseInt("dos") ?

    A. NumberFormatException
    B. InputMismatchException
    C. NullPointerException
    D. ArithmeticException

13. **Excepciones.** Un constructor hace throw new IllegalArgumentException("precio negativo"). Quien crea el objeto no usa try-catch. ¿Qué pasa?

    A. El objeto se crea con precio 0
    B. No compila
    C. El programa termina con la traza de la excepción
    D. La excepción se ignora

14. **Polimorfismo.** Perro y Gato heredan de Animal. ¿Qué pasa?

```
Animal a = new Gato();
Perro p = (Perro) a;
```

    A. No compila
    B. Compila y truena con ClassCastException
    C. Compila y convierte el gato en perro
    D. Compila y p queda en null

15. **Herencia.** ¿Qué efecto tiene declarar una clase como final?

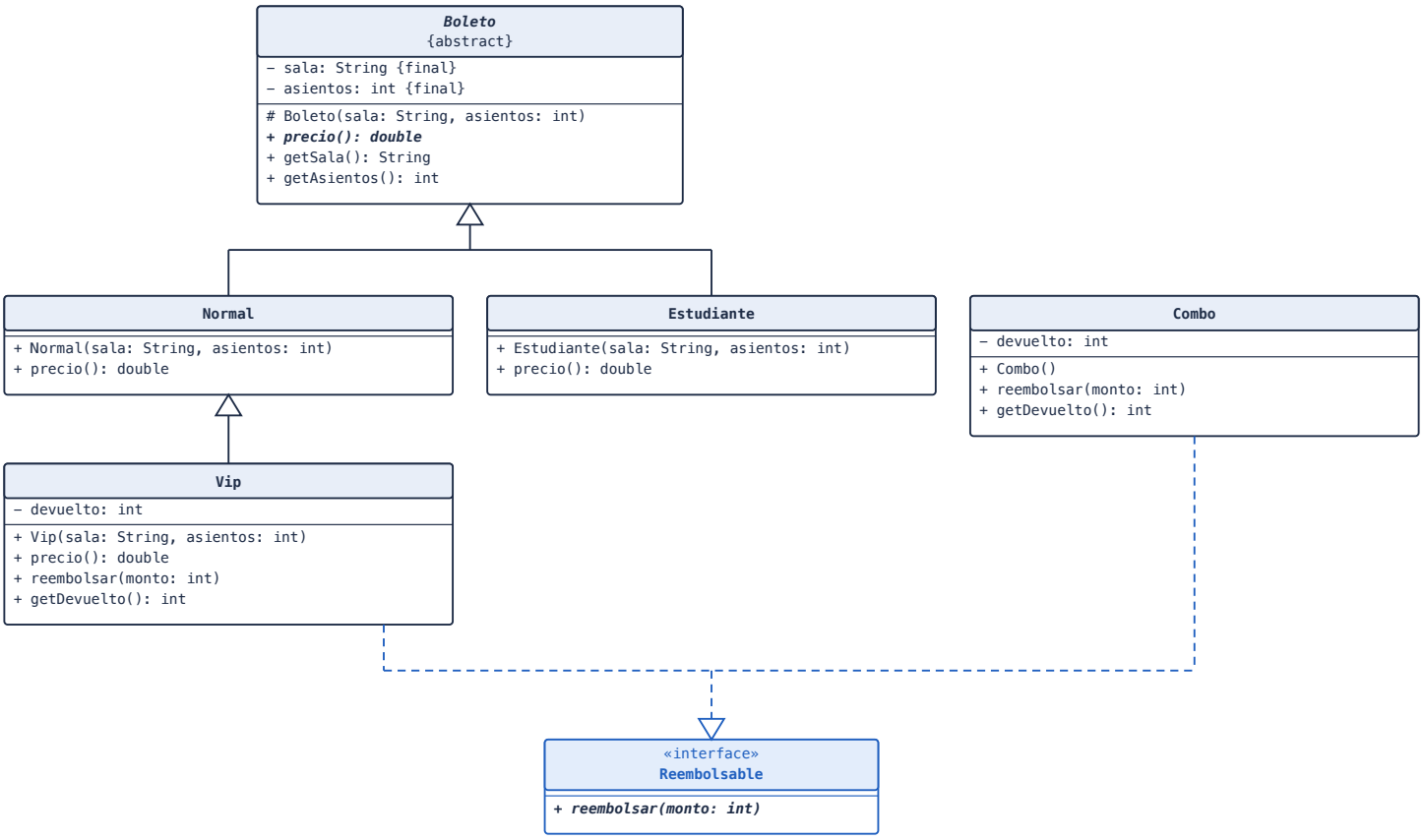
    A. No se pueden crear objetos de ella
    B. Sus atributos no pueden cambiar
    C. Todos sus métodos son abstractos
    D. Ninguna clase puede heredar de ella

16. **Herencia.** ¿Para qué sirve escribir @Override?

    A. Para que el compilador avise si en realidad no estás sobrescribiendo nada
    B. Para que el método corra más rápido
    C. Es obligatorio para heredar
    D. Para volver abstracto el método

4. Práctica de la Parte B · lee el diagrama

Un cine cobra por asiento cada boleto y además devuelve dinero de lo que sea reembolsable.



Linea continua con triangulo: extends. Punteada con triangulo: implements.
Cursiva: abstracto. En cada clase aparece solo lo que esa clase escribe; lo heredado no se repite.

Contesta con tus palabras, en una a tres oraciones:

1. ¿Cuáles tipos del diagrama no se pueden crear con new? Di por qué en cada caso.
2. Se ejecuta Boleto v = new Estudiante("Sala 2", 2); y luego v.precio(). ¿De qué clase es el método precio() que corre y por qué?
3. ¿Qué método está obligada a escribir la clase Combo y quién la obliga? ¿Qué pasa si no lo escribe?
4. ¿Se puede guardar un objeto Combo en un arreglo Boleto[]? ¿Y en un arreglo Reembolsable[]? Explica las dos respuestas.
5. sala es private en Boleto. ¿Cómo obtiene ese dato un objeto Estudiante? ¿Qué pilar se está aplicando y para qué sirve?
6. Lista todos los métodos que puedes llamar sobre un objeto Vip, incluidos los heredados, y di de dónde viene cada uno.

5. Práctica de la Parte C · escribe el código

Usa el diagrama del cine. Esta es la clase `Boleto`, ya escrita:

```
public abstract class Boleto {
    private final String sala;
    private final int asientos;

    protected Boleto(String sala, int asientos) {
        if (asientos <= 0) {
            throw new IllegalArgumentException("Los asientos deben ser mayores que cero");
        }
        this.sala = sala;
        this.asientos = asientos;
    }

    public abstract double precio();

    public String getSala() {
        return sala;
    }

    public int getAsientos() {
        return asientos;
    }
}
```

1. Escribe completa la clase Estudiante del diagrama. Un boleto de estudiante cuesta 50 pesos por asiento. El método precio() devuelve el valor con return, no lo imprime.
2. Escribe el método static double total(Boleto[] lote), que devuelve la suma de precio() de todos los boletos del arreglo. El arreglo puede traer normales, de estudiante y VIP en cualquier orden. No uses if ni instanceof para distinguir el tipo.
3. Para practicar interfaces: escribe completa la clase `Combo` del diagrama. Su método `reembolsar` suma el monto recibido a `devuelto`.

4. Para practicar excepciones: sin correrlo, ¿qué imprime este programa? Después explica por qué no se imprime `Creado`.

```
public class Demo {
    public static void main(String[] args) {
        Reembolsable[] cosas = { new Vip("Sala 5", 2), new Combo() };
        for (Reembolsable r : cosas) {
            r.reembolsar(40);
        }
        try {
            Boleto b = new Estudiante("Sala 2", 0);
            System.out.println("Creado");
        } catch (IllegalArgumentException ex) {
            System.out.println("No se pudo: " + ex.getMessage());
        } finally {
            System.out.println("Fin");
        }
    }
}
```